SPLUNK MACHINE LEARNING & AI

# AITK & DSDL — Usage Flow Guide

What each tool is for, how the data flows, and worked examples — from in-Splunk ML commands to containerized deep learning.

## 1. The big picture

Splunk's machine-learning stack is two complementary layers. You almost always start with AITK; you reach for DSDL only when you need heavy or custom models.

### AITK · Splunk AI Toolkit

*Formerly the Machine Learning Toolkit (MLTK).*

Runs ML **inside Splunk** using SPL. Ships ~30 classic algorithms (regression, clustering, classification, anomaly detection, forecasting), guided Assistants, and the `fit` / `apply` search commands. No containers, no Python to write.

**Best for:** tabular data, quick models, dashboards, well-known algorithms.

### DSDL · Data Science & Deep Learning

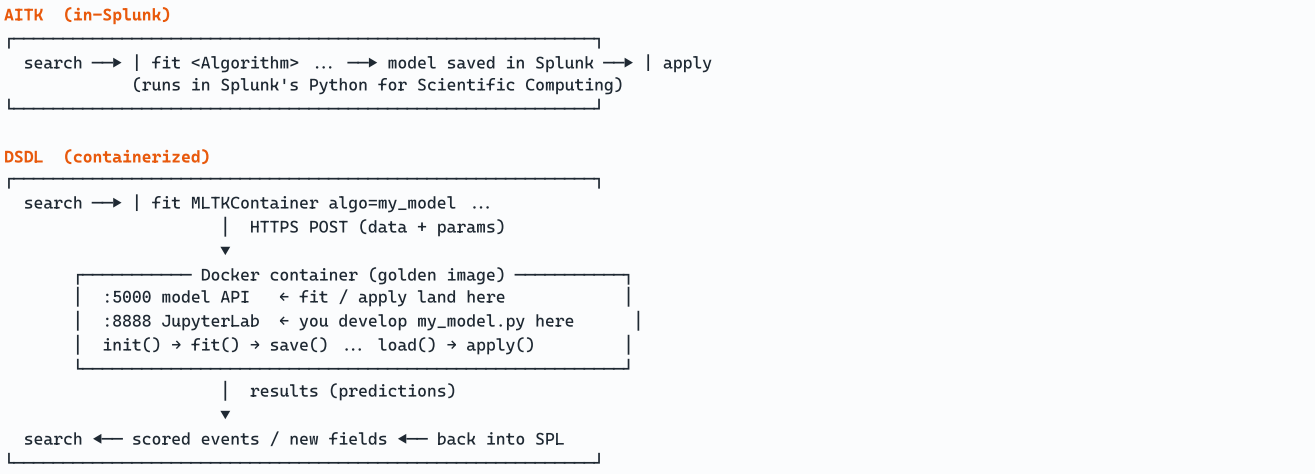
*Formerly the Deep Learning Toolkit (DLTK). Extends AITK.*

Runs your model in an **external Docker container** (TensorFlow, PyTorch, NLP/LLM libraries) with a JupyterLab dev environment. Splunk streams data to the container's API and gets predictions back.

**Best for:** neural nets, NLP/LLMs, GPU training, custom Python, anything beyond the built-in algorithms.

**Relationship:** DSDL is built on top of AITK. It adds the `MLTKContainer` algorithm to the same `fit` / `apply` SPL you already use in AITK — but the computation happens in the container instead of in Splunk. Install order is always **PSC → AITK → DSDL**.

## 2. How the data flows




Both expose the same verbs — `fit` (train) and `apply` (score). The only difference is **where the math runs**: in Splunk (AITK) or in the container (DSDL, via `algo=MLTKContainer`).

## 3. AITK vs DSDL — the difference in depth

Clear up the common misconception first. It is *not* "AITK = ready-made models you just use, DSDL = where you train." **Both train models on your own data using the same `fit` command.** The real difference is *how you get the model*: with AITK you pick an algorithm from a fixed catalog; with DSDL you write the model yourself in code.

**The one-line distinction —** AITK: choose from ~30 ready-made algorithms and train them on your data; you tune parameters, not the model design. DSDL: author any model from scratch in Python (any framework, any architecture) and run it in a container.

**AITK = order from the menu**

You pick a dish (algorithm) off the menu and the kitchen cooks it with your ingredients (data). Fast and easy — but you can only order what's on the menu, and you adjust seasoning (parameters), not the recipe itself.

**DSDL = cook in the kitchen**

You step into the kitchen and write the recipe yourself — any ingredients, any technique, any equipment (GPU). Total freedom at the cost of having to write and maintain the code and run the container.

### Detailed comparison

| ASPECT | AITK | DSDL |
|---|---|---|
| Trains on your data ( `fit` / `apply` ) | Yes | Yes |
| How you obtain a model | Pick one of ~30 built-in algorithms | Write it yourself in a Jupyter notebook |
| Algorithm catalog | Fixed (regression, classification, clustering, density/anomaly, forecasting, preprocessing) | Unlimited — whatever you code |
| Custom model architecture | No | Yes — define layers, loss, training loop |
| Frameworks under the hood | scikit-learn, StatsModels (fixed) | TensorFlow, PyTorch, HuggingFace, XGBoost, spaCy… anything `pip` -installable |
| Deep learning / neural nets | Very limited | Full |
| NLP / transformers / LLM / embeddings | No | Yes (incl. RAG) |
| GPU training | No | Yes (GPU golden image) |
| Custom preprocessing / feature engineering | Within SPL only | Arbitrary Python in the notebook |
| Where computation runs | Inside Splunk (PSC add-on) | External Docker container |
| Coding required | None — SPL only | Python (notebook) |
| Where the model is stored | In Splunk (model lookups) | In the container volume ( `/srv/app/model` ) |
| Setup & operations | Just install the app | Docker + container lifecycle + endpoint config |
| Latency / overhead | Low (in-process) | Network hop; container must be running |
| Extensibility | Limited (custom algos possible but constrained) | Effectively unlimited |
| Sweet spot | Classic ML on tabular/log data, dashboards, quick wins | DL, NLP/LLM, GPU, custom research |

**What only DSDL can do**

- Define your own network architecture (CNN / RNN / Transformer)
- Use PyTorch / TensorFlow / HuggingFace models
- Fine-tune or serve LLMs; build embeddings / RAG
- Train on a GPU
- Custom loss functions, custom training loops
- Arbitrary Python preprocessing + any `pip` package

**…but AITK wins when**

- A standard algorithm already fits the problem
- You want zero code and instant dashboards/Assistants
- You'd rather not run & maintain Docker
- You need low latency / simple operations
- The data is tabular and the model is "classic ML"

> **Rule of thumb:** try AITK first. If a built-in algorithm solves it, you're done with no code. Move to DSDL only when you hit the catalog's limits — e.g. our DGA detector is a char-level CNN, which AITK doesn't offer, so it lives in a DSDL notebook.

## 4. Using AITK — examples

AITK adds `fit` / `apply` plus helper commands. Models are saved in Splunk and reused by name. A few representative flows:

### 4.1 Forecasting / regression

```
| inputlookup server_power.csv
| fit LinearRegression "ac_power" from "total-cpu-utilization" "total-disk-utilization"
      into power_model
| apply power_model
| eval residual = 'ac_power' - 'predicted(ac_power)'
```

### 4.2 Anomaly detection (unsupervised)

```
index=web sourcetype=access_combined
| timechart span=5m count AS hits
| fit DensityFunction hits into hits_density
| apply hits_density
| where IsOutlier=1          ← flag unusual traffic spikes
```

### 4.3 Clustering

```
index=fw | stats sum(bytes) AS bytes count AS sessions by src_ip
| fit KMeans bytes sessions k=4 into ip_clusters
| apply ip_clusters          ← adds a "cluster" field per src_ip
```

### 4.4 Classification

```
| inputlookup firewall_traffic.csv
| fit RandomForestClassifier used_by_malware from src_bytes dest_bytes duration
      into fw_class
| apply fw_class
```

> **Assistants:** AITK also ships point-and-click Assistants (Predict Numeric/Categorical Fields, Detect Outliers, Forecast Time Series, Cluster Events) that generate this SPL for you — a great way to learn the commands.

## 5. Using DSDL — examples

DSDL routes `fit` / `apply` to a container running your model. You author the model once in JupyterLab; Splunk then calls it like any other algorithm.

### 5.1 The notebook contract

A DSDL model is a notebook whose cells are tagged with magic comments. On **save**, DSDL compiles the tagged cells into `/srv/app/model/<name>.py`:

| CELL TAG | FUNCTION | ROLE |
|---|---|---|
| `# mltkc_import` | `imports` | module top |
| `# mltkc_init` | `init(df, param)` | build the model |
| `# mltkc_fit` | `fit(model, df, param)` | train |
| `# mltkc_apply` | `apply(model, df, param)` | score → DataFrame |
| `# mltkc_save / _load` | `save() / load()` | persist / reload |
| `# mltkc_summary` | `summary(model)` | model info |

### 5.2 The development loop

1. **Stage** real data into the container (no training):
   `| inputlookup data.csv | fit MLTKContainer mode=stage algo=my_model ...`
2. **Iterate** in JupyterLab: `df, param = stage("my_model")` → run `init/fit/apply` by hand, tweak the cells.
3. **Save** the notebook → the `.py` is rebuilt.
4. **Train & score** from Splunk (no `mode=stage`):
   `... | fit MLTKContainer algo=my_model ... into app:my_model` then `... | apply my_model`.

### 5.3 Worked example — DGA domain detection on BOTSv1

A character-level neural network classifies algorithmically-generated malware domains vs benign ones, then scores real DNS queries from the BOTSv1 dataset.

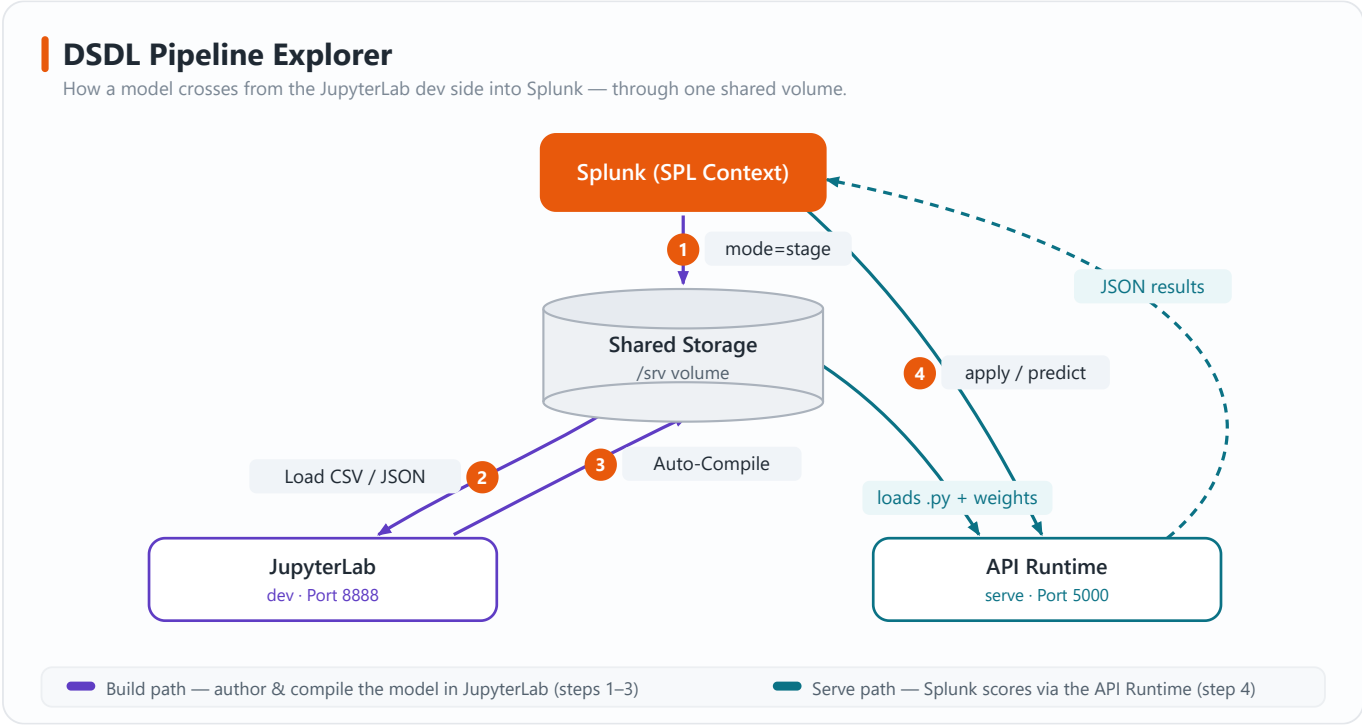
```
# 1) TRAIN – stream labeled domains to the container, train + save the model
| inputlookup dga_training_domains.csv
| fit MLTKContainer algo=dga_neural_network epochs=25 is_dga from domain
      into app:dga_model

# 2) SCORE – rank BOTSv1 DNS queries by how "DGA-like" they look
index=botsv1 sourcetype=stream:dns message_type=Query
| eval domain=lower(mvindex('query{}',0))
| stats count by domain
| apply dga_model
| where is_dga_predicted=1
| sort - dga_score
| table domain count dga_score
```

> **Why DSDL here?** A trainable char-level CNN (Keras/TensorFlow) isn't a built-in AITK algorithm — so the model lives in the container, while the SPL stays identical in spirit to AITK's `fit` / `apply`.

## 6. DSDL Pipeline Explorer — from JupyterLab into Splunk

How does the model you build in JupyterLab end up running inside Splunk? Through one thing they **both mount: the shared `/srv` volume**. You never export, upload or copy a model — when you **Save** the notebook, DSDL compiles it into that volume, and the API Runtime that Splunk talks to reads the very same files. Splunk only ever references the model by its `algo=` name.




DSDL Pipeline Explorer — the shared volume is the dev → prod bridge: JupyterLab (:8888) and the API Runtime (:5000) are two faces of the same container.

### The round trip, step by step

| STEP | WHAT HAPPENS | DIRECTION |
|---|---|---|
| **1. mode=stage** | Splunk pushes the search rows + params into the shared volume as CSV/JSON — *no training yet*. | Splunk → Volume |
| **2. Load CSV/JSON** | `df, param = stage("<model>")` loads exactly that data so you develop on the real input. | Volume → JupyterLab |
| **3. Auto-Compile** | On **Save**, the tagged `# mltkc_*` cells compile to `/srv/app/model/<model>.py`; training writes the **model weights** back to the volume. | JupyterLab → Volume |
| **4. apply / predict** | `\| fit/apply MLTKContainer algo=<model>` — the API Runtime loads that `.py` + weights and returns **JSON results** into SPL. | Splunk ⇄ API Runtime |

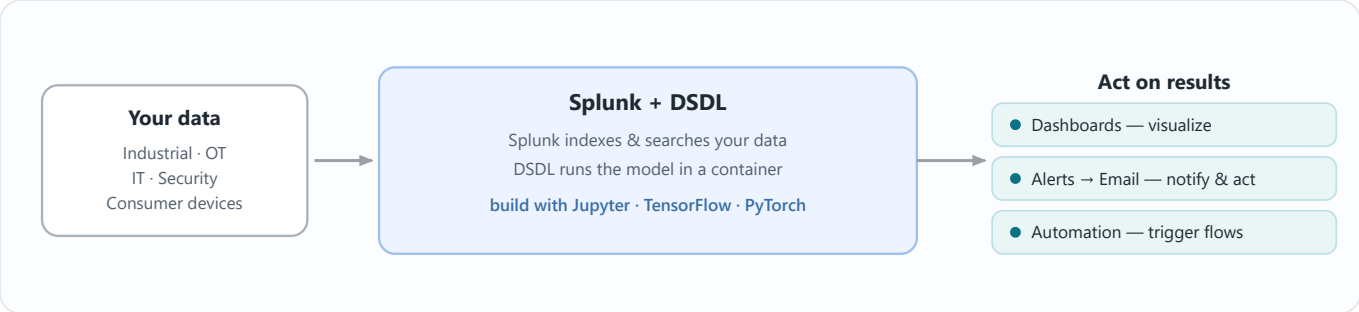
> **Why this matters:** "deploying" a DSDL model is just **Save the notebook** — there is no separate publish step. The next `fit` / `apply` from Splunk already runs your latest compiled code and weights, because the API Runtime reads the same volume JupyterLab wrote to. In this lab both the dev (:8888) and serving (:5000) faces run inside the one compose service `mltk-dev` (DEV mode).

## 7. DSDL architecture — the official picture

Zooming out from this lab: this is the architecture Splunk documents for DSDL. It's the same dev → serve idea as section 6, shown at full scale — many production models at once, your choice of container orchestrator, and optional monitoring. Plain-language walkthrough below.

### 7.1 Where DSDL fits in the workflow

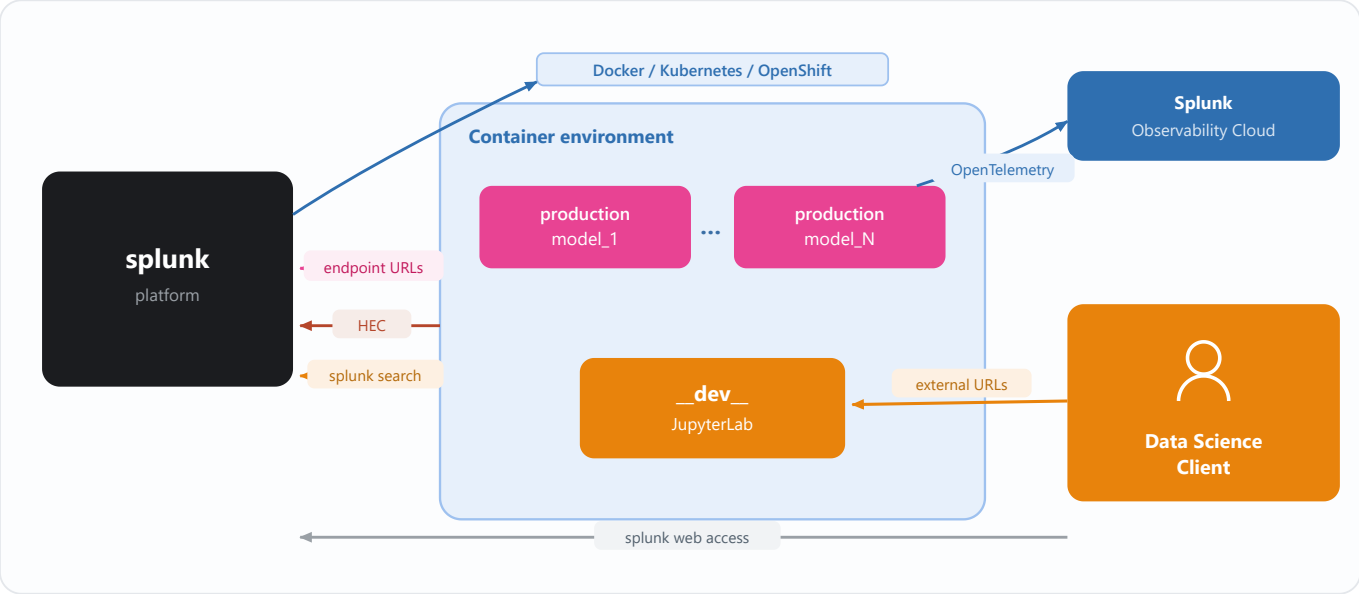
Any data — industrial/OT, IT, security, devices — lands in Splunk. You build and run the model in DSDL's containers, and Splunk turns the scored results into something a human (or another system) can act on.




Where DSDL fits — data in → model in containers → act on the results.

### 7.2 How DSDL connects to Splunk

DSDL links your **Splunk platform** to a **container environment** (Docker, Kubernetes or OpenShift). It starts and stops containers on demand and moves data across a few well-defined channels — explained under the diagram.




How DSDL connects Splunk to the container environment — production models (model_1...model_N), one `__dev__` JupyterLab, plus the data scientist and optional Observability.

| CHANNEL | WHAT IT'S FOR (PLAIN WORDS) | DIRECTION |
|---|---|---|
| **Container runtime**<br>Docker / K8s / OpenShift | DSDL starts & stops dev and production containers on demand. | Splunk → containers |
| **Endpoint URLs** | The main pipe — `fit` / `apply` sends data out and gets predictions back.<br>*This is what the lab uses.* | Splunk ⇄ containers |
| **splunk search** (REST API) | Optional — pull ad-hoc search results straight into Jupyter to experiment. | container → Splunk |
| **HEC** | Optional — a container pushes events/results directly into a Splunk index. | container → Splunk |
| **External URLs** | You (the data scientist) open JupyterLab / TensorBoard / MLflow in a browser. | you → containers |
| **OpenTelemetry** → Observability | Optional — container CPU/memory + endpoint metrics flow to Splunk Observability Cloud. | containers → Observability |

> **Dev vs production containers:** the `__dev__` container is where you build — one interactive JupyterLab. **Production** containers are lean serving copies (`model_1` … `model_N`) that just run `apply` for live searches, and you can run many at once. In this POC lab a *single* `mltk-dev` container over Docker plays both roles — Kubernetes/OpenShift and Observability are the very same picture scaled up.

## 8. End-to-end flow (this lab)

```
1. Install          PSC  →  AITK  →  DSDL   (Splunk apps)
2. Configure DSDL   Setup page → Docker mode → Test & Save
3. Container        golden image runs as compose service "mltk-dev" (DEV mode)
                    :5000 model API   :8888 JupyterLab   :6006 TensorBoard
4. Develop          open https://localhost:8888  → edit notebook → Save
5. Train            | fit MLTKContainer algo=...  into app:model
6. Score / detect   | apply model   → dashboards, alerts, notable events
```

### Key facts for this environment

| ITEM | VALUE |
|---|---|
| Splunk Web | `http://localhost:8000` (admin / your password) |
| JupyterLab | `https://localhost:8888` — HTTPS, password `splunkdsdl` |
| Model API (fit/apply) | `https://host.docker.internal:5000` (Endpoint URL in DSDL) |
| DSDL Docker Host | `tcp://docker-proxy:2375` (sidecar; Splunk can't read the raw socket) |
| Container | compose service `mltk-dev` — do not also start one in the DSDL UI |

### Common gotchas

- JupyterLab is **HTTPS** — `http://localhost:8888` returns "didn't send any data". Use `https://` and accept the self-signed cert.
- Container must be **running** for `fit` / `apply` to work.
- Edited a notebook but the search runs old code? You forgot to **Save** (the `.py` only regenerates on save).
- **Check Hostname = Disabled** in DSDL Setup (self-signed dev certs).

## 9. Quick decision guide

| YOUR SITUATION | USE |
|---|---|
| "Find anomalies / forecast / cluster my logs" | **AITK** built-in algorithm or Assistant |
| "Predict a number/category from tabular fields" | **AITK** regression / classification |
| "I need a neural net / transformer / embeddings / LLM" | **DSDL** custom notebook |
| "I have Python model code / a specific framework" | **DSDL** container |
| "I need a GPU to train" | **DSDL** (GPU golden image) |